

TP n°1

Langage Procédural : C

Écriture d'un parseur simple SGML

L'objectif de ce TP est de construire un *parseur* de fichier SGML simple. Ce parseur devra être capable de reconnaître si un langage balisé donné est bien formé.

1 Fichier SGML

Le modèle SGML (*Standard Generalized Markup Language*) est un standard qui a pour la première fois permis la spécification de la structure d'un langage à balise, sur lequel sont contruits bon nombre de langages (HTML, XML, ...).

1.1 Définition du fichier

Basiquement, un fichier SGML définit une structure de document sous forme d'arbre. Chaque Nœud de l'arbre formé peut posséder des attributs et porter une valeur. Un nœud est défini par une balise ouvrante (`<nom_balise>`) et une balise fermante (`</nom_balise>`) pour lesquelles le nom de la balise est identique.

Chaque balise ouvrante peut posséder zéro ou plusieurs attributs. Ils sont spécifiés de la manière suivante : `<nom_balise attr1=val1 attr2=val2 ...>`. Les valeurs de chaque attribut peuvent être indiquées sous forme d'une chaîne de caractère (`'val'` ou `"val"`) ou d'un texte libre (le séparateur indiquant la fin de chaîne sera alors le caractère espace `' '`). Le texte indiqué entre une balise ouvrante et fermante sera considéré comme étant la valeur associée au nœud.

Enfin, la structure de nœuds formée par les balises ouvrantes et fermantes doit correspondre à la description d'un arbre (i.e. les balises doivent être imbriquées et ne pas se chevaucher comme indiqué dans les figures 1 et 2.

Listing 1 – Structure arborescente valide

```
<B1>
  <B2>
    </B2>
  </B1>
```

Listing 2 – Structure arborescente non valide

```
<B1>
  <B2>
</B1>
  </B2>
```

La figure 3 montre une syntaxe valide de fichier SGML.

Un fichier SGML se définit dans la norme comme possédant 3 composantes :

Listing 3 – Exemple de code SGML

```
<H>Titre A de niveau a
  <P>Introduction</P>
  <H>Titre A de niveau b
    <P>Description des éléments du titre A de niveau b</P>
  </H>
  <H>Titre B de niveau b
    <P>Description des éléments du titre B de niveau b</P>

    <H>Titre A de niveau c
      <P>Description des éléments du titre A de niveau c</P>
    </H>
  </H>
</H>
<H>Titre B de niveau a
  <P>Conclusion</P>
</H>
```

- Sa structure qui sera traitée par des DTD (document définissant la nature, le nom des balises et leurs attributs possibles)
- Sa présentation qui sera traitée par des feuilles de style
- Ses données qui seront rédigées dans des instances (document balisés selon une DTD)

Mais dans ce TP, nous nous proposons uniquement de vérifier la syntaxe du langage balisé. En conséquence, nous ne prendrons pas en considération les feuilles de style ni les DTD. Le travail consistera à analyser le fichier afin de vérifier que le document contient uniquement des balises au sens SGML.

1.2 Lecture du fichier

Pour lire un fichier, vous devez utiliser les fonctions fournies par la librairie standard du C. Ces fonctions sont regroupées dans le fichier d'entête `<stdio.h>`.

Afin d'accéder à un fichier, vous devez au préalable obtenir un accès vers ce fichier (l'ouvrir) à l'aide de la fonction `fopen` dont la signature est `FILE* fopen(const char *filename, const char* mode);`. Cette fonction prends 2 chaînes de caractères en paramètres, la première indiquant le chemin vers le fichier à ouvrir, et la deuxième les droits d'accès à y appliquer (lecture, écriture, ...). Cette fonction retourne un pointeur vers un type `FILE` qui sera par la suite utilisé comme intermédiaire pour accéder au fichier. Si la valeur de ce pointeur est égale à `NULL`, le fichier n'a pu être ouvert correctement.

Pour effectuer une opération de lecture sur un fichier correctement ouvert, vous devrez utiliser dans ce TP la fonction `fgetc` dont la signature est `int fgetc(FILE *stream);`. Cette fonction prend en paramètre le pointeur retourné par la fonction `fopen` et retourne le code ASCII du caractère courant du fichier, puis incrémente la position de lecture sur le caractère suivant dans le fichier.

Vous pouvez vérifier si vous avez atteint la fin du fichier en utilisant la fonction `feof` de signature `int feof(FILE *stream);`. Si la valeur retournée par cette fonction est différente de 0, la fin du fichier a été atteinte lors de la dernière opération de lecture.

Enfin, vous devez clore le fichier correctement en appelant la fonction `fclose` dont la signature est `int fclose(FILE *stream);`.

La figure 4 donne un exemple de lecture d'un fichier caractère par caractère.

Listing 4 – Exemple de lecture d'un fichier caractère par caractère

```
#include <stdio.h>

int main ()
{
    FILE *pFile;
    char c;

    if (pFile=fopen("myfile.txt","rt"))
    {
        for (c=fgetc(pFile);!feof(pFile);c=fgetc(pFile))
        {
            ...
        }
        fclose (pFile);
    }
    else printf("erreur_d'ouverture_du_fichier\n");
    return 0;
}
```

2 Construction du parseur

Veuillez penser à créer dans votre programme des fichiers source (.c) et des fichiers entête correspondant (.h) dans lesquels vous déclarerez les fonctions que vous créerez. Pensez à bien découper votre programme en fonctions afin de le rendre plus lisible et plus facilement maintenable.

Vous compilerez vos sources à l'aide de `gcc` sous Linux en utilisant l'option `-Wall` permettant d'afficher tous les *warnings* lors de la compilation.

2.1 Analyse lexicale

Dans un premier temps, vous allez créer un automate qui va lire caractère par caractère un fichier en détectant les différentes balises du fichier et en récupérant les noms de la balise retournée.

2.1.1 Automates à états finis

Un automate à états finis est une machine abstraite utilisée notamment dans la reconnaissance de langages syntaxiques. Une telle machine possède une mémoire finie permettant de stocker l'état interne de l'automate. Les machines à états finis sont étudiées dans la théorie des automates, un sous-domaine de l'informatique théorique.

2.1.2 Modélisation algorithmique

Cette machine peut être modélisée algorithmiquement comme un graphe orienté dont les nœuds représentent un nombre fini d'états. Chaque état est associé à une étape de reconnaissance du langage. Les arcs du graphe correspondent à des transitions d'un état interne de l'automate vers un nouvel état. Il peut exister, depuis un état, zéro, une ou plusieurs transitions vers d'autres états. On distingue un état initial et potentiellement plusieurs états finaux.

Pour un état donné, on applique une transition vers un nouvel état en fonction d'un contexte dépendant de l'application traitée. Dans le problème qui nous préoccupe, nous cherchons à analyser une suite de caractères. Cette analyse s'effectue en lisant la chaîne caractère par caractère. À chaque lecture d'un nouveau caractère, l'état interne de l'automate correspond à la mémoire (ou à l'état de

la mémoire) de l'analyseur syntaxique et le contexte de l'automate sera représenté par le caractère actuellement analysé. La transition suivie sera donc déterminée, pour un état donné, par la valeur du caractère lu. S'il n'existe pas de transition possible pour un état donné, la nature de cet état, final ou non, déterminera respectivement la reconnaissance ou non du langage par l'automate.

La figure 1 montre un exemple simple d'automate reconnaissant une fonction basique (sans prise en compte des paramètres ni de valeur de retour) écrite en pseudo-C autorisant un espace optionnel après le nom de la fonction.

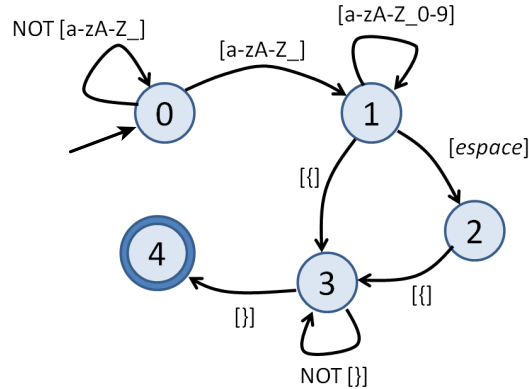


FIGURE 1 – Exemple d'automate reconnaissant une fonction.

2.1.3 Implémentation efficace d'un automate à états finis

Afin de rendre le code produit le plus lisible possible, il est important de regrouper toutes les opérations d'un même type dans un même endroit du source que vous écrivez.

Dans le cas qui nous préoccupe ici (reconnaissance de langage par un automate fini), il est judicieux de ne lire un nouveau caractère qu'à un unique endroit du code. Cela permet de garder lisible directement la logique de transition des états et de simplifier la maintenance de l'application. La figure 5 montre un exemple de squelette d'implémentation d'un tel automate.

Vous pouvez par exemple créer une fonction qui extrait la prochaine balise contenue dans le fichier, en signifiant s'il s'agit d'une balise ouvrante ou fermante.

2.2 Analyse syntaxique

Dans un deuxième temps, vous devez vérifier la structure arborescente de l'organisation des balises.

Vous pouvez par exemple utiliser un tableau organisé sous forme de pile, dans lequel vous stockez une nouvelle balise dès que vous rencontrez une balise ouvrante.

Lorsque vous rencontrez une balise fermante, si le dernier nom inséré dans le tableau correspond au nom de la balise fermante, cela signifie que la syntaxe est valide pour cette balise et vous pouvez supprimer le nom de la balise de la pile. Sinon, cela signifie que la structure des balises ne correspond pas à un arbre, et vous pouvez stopper l'exécution du programme en renvoyant un message d'erreur sur la console.

Listing 5 – Structure générale de codage d'un automate à état fini

```
enum Etats
{
    SEtatDebut,
    SEtat1,
    ...
};

enum Etats curEtat; // Etat courant de l'automate
char c; // caractère courant analysé

curEtat=SEtatDebut;
while (condition_de_sortie)
{
    // Récupérer un caractère
    c=fgetc(fp);

    // Calcul des transitions
    switch(curEtat)
    {
        case SEtatDebut: // Automate dans l'état initial
            ...
            if (...) curEtat=SEtat1; // Exemple de transition
            break;

        case SEtat1: // Automate dans l'état 1
            ...
            break;

        ...
    }
}
```